

# CLASSIFICATION OF HAND-WRITTEN DIGITS

ÁLVARO GALISTEO BERMÚDEZ, DANIEL RATH

## CONTENTS

|                                                       |   |
|-------------------------------------------------------|---|
| 1. Logistic regression for multi-class classification | 1 |
| 1.1. Our problem                                      | 1 |
| 1.2. Loss function                                    | 2 |
| 1.3. Solvability                                      | 2 |
| 1.4. Regularization                                   | 3 |
| 2. Stochastic gradient descent                        | 4 |
| References                                            | 5 |

## 1. LOGISTIC REGRESSION FOR MULTI-CLASS CLASSIFICATION

Logistic regression is a commonly used model for classification tasks. In our case we want to use logistic regression for multi-class classification, which is a generalization of the binary logistic regression.

In our case, we want to predict the class of a given  $8 \times 8$  grayscale image of a hand written digit. One would expect that a given input can not be classified as more than one digit at the same time. Therefore, we want to compute the probability of a given input image to belong to every of the 10 classes (digits from 0 to 9) and assign the class with the highest probability to the input image.

### 1.1. Our problem.

An input image is represented as a vector  $a_j \in \mathbb{R}^{64}$ . Each entry of the vector is an integer between 0 and 16 and corresponds to the intensity of a pixel in the image with 0 being the lowest intensity and 16 being the highest intensity. Each image  $a_j$  in the dataset is associated with a label  $y_j \in \{0, \dots, 9\}$ , which is the digit that is represented in the image. In order to use the labels in our model, we encode them into one-hot vectors  $p_j \in \{0, 1\}^{10}$ , where the  $i$ -th entry is 1 if the label is  $i$  and 0 otherwise. [Dev]

We want to train a function  $\varphi : \mathbb{R}^{64} \times (\mathbb{R}^{64} \times \mathbb{R})^{10} \rightarrow \mathbb{R}^{10}$  that takes an input image  $a_j$  and a parameter  $x = (w_i, b_i)_{i=0}^9$  and outputs a vector of scores for each class. The score for class  $i$  is computed as  $\varphi(a_j; x)_i = a_j^T w_i + b_i$ . The  $w_i \in \mathbb{R}^{64}$  is a weight vector and  $b_i \in \mathbb{R}$  is a bias term for class  $i$ .

The training of the model consists of the following optimization problem

$$(1.1) \quad \underset{x \in (\mathbb{R}^{64} \times \mathbb{R})^{10}}{\text{minimize}} \quad \frac{1}{N} \sum_{j=1}^N L(p_j, \varphi(a_j; x)),$$

with  $N \in \mathbb{N}$  being the number of training samples and  $L$  the cross entropy loss defined as

$$(1.2) \quad L(p, s) = \log \left( \sum_{l=0}^9 \exp(s_l) \right) - \sum_{i=0}^9 p_i s_i.$$

After the model has been trained, we can use it to classify new input images  $a$  by computing  $\varphi(a; x)$  and assigning the class with the highest score to the input image, i.e.

$$\tilde{y} = \arg \max_{i \in \{0, \dots, 9\}} \varphi(a; x)_i.$$

For logistic regression the scores in  $\varphi(a; x)_i$  are interpreted as probabilities by applying the softmax function, i.e.

$$(1.3) \quad \sigma(s)_i = \frac{\exp(s_i)}{\sum_{l=0}^9 \exp(s_l)}.$$

Note that in order to classify an input image, we do not need to compute the probabilities  $\sigma(\varphi(a; x))_i$  explicitly. This is because the softmax function is monotonically increasing in each of its entries. Therefore, the class with the highest score is also the class with the highest probability. However, we will see in the next section that the softmax function is important for the choice of our loss function.

### 1.2. Loss function.

We want to briefly discuss the choice of the loss function (1.2). For this sections we used [GBC16, Section 6.2.2.3] as a reference. In general, for a given input image  $a_j$  we want to predict the probabilities  $\sigma(\varphi(a_j; x))_i$  to belong to each of the classes  $0 \leq i \leq 9$ .

For this, we want our model to maximize the likelihood to observe the training data. If we assume each training image to be independent, the likelihood that our model predicts the observed training data correctly is given as

$$\mathcal{L}(x) := \prod_{j=1}^N \sigma(\varphi(a_j; x))_{y_j}.$$

In order to deal with sums instead of products, we can equivalently minimize

$$(1.4) \quad -\log \mathcal{L}(x) = -\sum_{j=1}^N \log \sigma(\varphi(a_j; x))_{y_j}.$$

If we now use the definition of the softmax function (1.3), we can write

$$\begin{aligned} -\log(\sigma(\varphi(a_j; x))_{y_j}) &= -\log\left(\frac{\exp(\varphi(a_j; x)_{y_j})}{\sum_{l=0}^9 \exp(\varphi(a_j; x)_l)}\right) \\ &= \log\left(\sum_{l=0}^9 \exp(\varphi(a_j; x)_l)\right) - \varphi(a_j; x)_{y_j}. \end{aligned}$$

The last expression is identical to the loss function in (1.2), when substituting  $\varphi(a_j; x)_i$  with  $s_i$ . Therefore, minimizing (1.4) yields the same result as solving our optimization problem in (1.1). Also one can see that the choice of the softmax function to get the probabilities from the scores is crucial for the choice of our loss function.

### 1.3. Solvability.

The given optimization problem is a convex problem. This is because the log-sum-exp function is convex as we have already proved in the lecture. For reference one can also check [Boy09, pp. 72, 74]. Also the second term of our loss function (1.2) is linear and therefore convex. Also,  $\varphi$  is an affine mapping in  $x$ . And at the same time a convex function over an affine function is also convex (cf. [Boy09, p. 67]).

In general however, the optimization problem is not necessarily solvable. This is because the loss function (1.2) is not strictly convex in its scores  $s$  and therefore not strictly convex in the parameter  $x$ .

This, is because the loss function (1.2) is invariant under the addition of the same constant to every score  $s_i$ . More precisely, for a constant  $c \in \mathbb{R}$  we have

$$\log\left(\sum_{l=0}^9 \exp(s_l + c)\right) = \log\left(\exp(c) \sum_{l=0}^9 \exp(s_l)\right) = \log\left(\sum_{l=0}^9 \exp(s_l)\right) + c$$

and also

$$\sum_{i=0}^9 p_i(s_i + c) = \sum_{i=0}^9 p_i s_i + c.$$

Therefore, we have

$$L(p, s + c\mathbb{1}) = \log\left(\sum_{l=0}^9 \exp(s_l + c)\right) - \sum_{i=0}^9 p_i(s_i + c) = L(p, s) + c - c = L(p, s).$$

This implies that for a solution  $x$  of our optimization problem, we get arbitrary additional solutions, e.g.  $x' = (w_i + c\mathbf{1}, b_i)_{i=0}^9$  for any constant  $c \in \mathbb{R}$ . Therefore, the optimization problem is not uniquely solvable.

Even if we ignore non-uniqueness, the minimum in general may fail to be attained at all. A reason is linear separability of the training data. For a parameter  $x$  and

$$s_{j,i} := \varphi(a_j; x)_i = a_j^\top w_i + b_i,$$

and  $p_{j,y_j} = 1$ , we can rewrite the loss as

$$(1.5) \quad L(p_j, s_j) = \log \left( \sum_{l=0}^9 e^{s_{j,l}} \right) - s_{j,y_j} = \log \left( 1 + \sum_{i \neq y_j} e^{-(s_{j,y_j} - s_{j,i})} \right) \geq 0.$$

Hence  $L(p_j, s_j)$  gets small when all the  $s_{j,y_j} - s_{j,i}$  are large, e.g. when the correct class score  $s_{j,y_j}$  dominates all other scores  $s_{j,i}$  by a large margin  $\gamma > 0$ . If there exists a parameter  $\bar{x} = (\bar{w}_i, \bar{b}_i)_{i=0}^9$  such that this is the case, we call the dataset *separable*.

We are then able to scale the parameter  $\bar{x}$  with a factor  $t > 0$

$$\bar{x}^{(t)} := (t\bar{w}_i, t\bar{b}_i)_{i=0}^9.$$

and get  $s_{j,i}^{(t)} = \varphi(a_j; x^{(t)})_i = t s_{j,i}$ . Thus, for the differences we then get

$$s_{j,y_j}^{(t)} - s_{j,i}^{(t)} = t(s_{j,y_j} - s_{j,i}) \geq t\gamma.$$

If we use this, the above expression (1.5) yields

$$0 \leq L(p_j, \varphi(a_j; \bar{x}^{(t)})) = \log \left( 1 + \sum_{i \neq y_j} e^{-t(s_{j,y_j} - s_{j,i})} \right) \leq \log(1 + 9e^{-t\gamma}) \xrightarrow[t \rightarrow \infty]{} 0.$$

Therefore, if there exists such a separating parameter  $\bar{x}$ , the optimization term  $\sum_{j=1}^N L(p_j, \varphi(a_j; \bar{x}^{(t)}))$  is going to be arbitrarily small for large  $t$ , i.e. for large weights and biases  $t\bar{w}_i$  and  $t\bar{b}_i$ . This implies that the minimum of our optimization problem is not attained at a finite parameter  $x$  and therefore the optimization problem is not solvable. **Explain what is happening in the separable case from a geometric point of view!**

*Remark 1.3.1.* The idea, that linear separability can be a problem for logistic regression, was inspired by [Ada18, Linear Separability and Regularization].

#### 1.4. Regularization.

solves some problems **explain why regularization is a good thing to do!**

**It is fairly obvious that the optimization problem is  $C^\infty$ . Where should we put this?**

## 2. STOCHASTIC GRADIENT DESCENT

In order to solve the optimization problem of our logistic regression model, we use stochastic gradient descent.

## REFERENCES

- [Ada18] Ryan Adams. *Linear Classification with Logistic Regression. Lecture notes*. COS 324 – Elements of Machine Learning. Princeton University. Oct. 8, 2018. URL: <https://www.cs.princeton.edu/courses/archive/spring19/cos324/files/logistic-regression.pdf> (visited on 02/10/2026).
- [Boy09] Stephen P. Boyd. *Convex optimization*. Ed. by Lieven Vandenberghe. 7th ed. first published 2004. Cambridge University Press, 2009. 716 pp. ISBN: 9780521833783.
- [Dev] scikit-learn Developers. *Documentation of scikit-learn Machine Learning in Python*. Version 1.8.0 (stable). URL: <https://scikit-learn.org> (visited on 02/09/2026).
- [GBC16] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. <http://www.deeplearningbook.org>. MIT Press, 2016.